kjob

在 Notebook 裡，像國網一樣提交 GPU 工作

安裝 · 提交 · 監看 · 叢集狀態 · AI 助手

```
$ ksinfo
$ ksub -G 1 -g v100 train.py
Submitted job train-3f2a9c
$ klogs train-3f2a9c -f
epoch 1/50  loss 2.31
epoch 2/50  loss 1.87
…
```

github.com/MarkJiang-maji/kjob · branch: cgu-public

kjob 做了什麼

把 Notebook 裡的程式交給叢集執行，環境完全一樣

1

Notebook 寫程式

`ksub train.py`

2

叢集排程

等到指定的 GPU 有空

3

GPU 節點執行

同 image 、同路徑、同磁碟

4

結果回到 Notebook

~/kjob-logs/ 與你的輸出檔

Notebook 不必占 GPU

訓練交給工作，跑完就釋放 GPU 給別人

路徑不用改

/home/jovyan 、群組資料夾在工作裡位置相同

關掉瀏覽器也沒關係

輸出即時存檔，隨時用 klogs 查看

會用國網，就會用 kjob

指令名稱與用法對照

國網 / Slurm	kjob	用途
`sbatch`	`ksub`	提交後立即返回，工作在背景執行
`srun`	`ksrun`	提交並即時看輸出， Ctrl-C 取消
`squeue -u $USER`	`kqueue`	自己的工作
`squeue`	`kqueue -A`	全叢集的 GPU 工作與排隊原因
`sinfo`	`ksinfo`	每種 GPU：總數、使用中、空閒、排隊
`scontrol show job`	`kshow NAME`	單一工作的詳細資訊與事件
`sacct`	`kacct`	提交紀錄、結束代碼、執行時間
`srun --jobid --pty`	`kexec NAME [指令]`	進到執行中的工作：跑指令或開 shell
`scancel`	`kcancel NAME`	取消工作
`#SBATCH`	`#SBATCH / #KJOB`	腳本開頭寫資源設定

安裝：在 Notebook 的 Terminal

JupyterLab： File → New → Terminal；遠端桌面：開終端機

```
$ python3 -m pip install --user --upgrade \
      https://github.com/MarkJiang-maji/kjob/archive/refs/heads/cgu-public.tar.gz
$ export PATH="$HOME/.local/bin:$PATH"
$ echo 'export PATH="$HOME/.local/bin:$PATH"' >> ~/.bashrc  # 下次開 Terminal 也有效
$ kinfo                                                     # 確認安裝成功
```

Python 3.8 以上

pip 會自動安裝 kubernetes 套件，不會動到 PyTorch

裝在 ~/.local

在你的工作磁碟上， Notebook 重開仍然在

更新與虛擬環境

更新：重跑第一行。在虛擬環境內就拿掉 --user

提交前：先看環境與空的 GPU

kinfo 看自己， ksinfo 看叢集

```
$ kinfo
notebook  mynb-0  (namespace m1461023)      image  cguaicadmin/newlab-newpytorch:V1.0.8
  /home/jovyan             nfs 10.100.1.35:/Public/...   # 工作看得到的磁碟
  requests.nvidia.com/gpu  0 / 8                         # GPU 配額（Notebook 也算）
$ ksinfo
GPU_TYPE              NODES  GPUS  USED  FREE  WAITING_GPUS  GPU_MEM  NOTE
NVIDIA-RTX-A5000      1      10    10    0     -             24GB
Tesla-V100-SXM2-16GB  4      32    2     30    8 (1 jobs)    16GB
rtxpro6000            1      8     4     4     -             -        restricted
```

FREE

現在可用的 GPU 數，選有空位的型號

WAITING_GPUS

正在排隊等這種 GPU 的需求

restricted

需管理員開通；沒開通會一直排隊

第一個 GPU 工作： ksrun

提交、即時看輸出、結束後自動清理

```
$ ksrun -G 1 -g v100 -c 2 --mem 8Gi gpu_test.py
kjob: submitted gpu-test-1a2b3c
kjob: gpu-test-1a2b3c running on hs5800-1
[kjob 03:12:05Z] start gpu-test-1a2b3c on hs5800-1
node: hs5800-1 | GPU: Tesla V100-SXM2-16GB
step 1/5 ok
…
step 5/5 ok
kjob: gpu-test-1a2b3c Succeeded (exit code 0)
```

`-G 1`

使用 1 張 GPU

`-g v100`

GPU 型號，名稱看 ksinfo

`-c 2 --mem 8Gi`

CPU 核心與記憶體

`Ctrl-C`

取消工作（ksrun 專屬）

長時間訓練： ksub 背景提交

提交後就可以關掉瀏覽器

```
$ ksub -J resnet -G 1 -g v100 -c 4 --mem 32Gi --shm 8Gi -t 12h train.py --epochs 50
Submitted job resnet  (log: /home/jovyan/kjob-logs/resnet.log)
```

選項	意思（預設）
`-J NAME`	工作名稱（腳本名 + 亂碼）
`-G N`	GPU 數（ 1； 0 = 只用 CPU ）
`-g TYPE`	GPU 型號（任何一種）
`-c N`	CPU 核心（ 1 ）
`--mem 32Gi`	記憶體（ 2Gi ）

選項	意思（預設）
`--shm 8Gi`	/dev/shm （ 256Mi ）
`-t 12h`	時間上限（無）
`-w NODE`	指定節點
`--python PATH`	指定 Python ／虛擬環境
`-D / -o`	工作目錄／日誌目錄

時間格式： 30 （分）、 2:00:00 、 1-12:00:00 （天 - 時 : 分 : 秒）、 12h 、 2d　完整選項： docs/USER_GUIDE.md

查看自己的工作

工作在背景跑，這些指令隨時可用

1	`kqueue`	自己的工作：狀態、節點、 GPU
2	`klogs NAME -f`	即時看輸出（ Ctrl-C 只停止觀看）
3	`kshow NAME`	詳細資訊；排隊太久看這裡的原因
4	`kwait NAME`	等到結束，回傳程式的結束代碼
5	`kexec NAME`	在工作裡跑指令（如 nvidia-smi ）或開 shell
6	`kacct`	歷史紀錄：成功與否、跑了多久

```
$ kqueue
NAME    STATE    NODE      GPU
resnet  Running  hs5800-1  1

$ kacct
NAME    GPU  STATE      EXIT
demo1   1    COMPLETED  0
resnet  1    RUNNING    -
probe   1    FAILED     137
```

整個叢集：誰在用、誰在排隊

例如想知道 RTX PRO 6000 現在的狀況

```
$ ksinfo -g rtxpro6000
NODE         GPU_TYPE    GPU(used/total)  CPU(req/total)  MEM_GiB(req/total)  STATE
rtxpro6000   rtxpro6000   4/8              20.1/256        160.1/2015.4        up
$ kqueue -A --state pending
USER       KIND      NAME   STATE    GPU  GPU_TYPE               NODE  TIME  WAITING_FOR
d1400702   Notebook  brain  Pending  8    Tesla-V100-SXM2-16GB   -     2h    Insufficient nvidia.com/gpu
*m1461023  kjob      exp3   Pending  2    NVIDIA-RTX-A5000       -     5m    Insufficient nvidia.com/gpu
running: 0 workloads / 0 GPUs;  pending: 2 workloads / 10 GPUs   (* = your namespace)
```

`kqueue -A`

所有使用者的 GPU 工作， USER 是 namespace

`-u / -g / --state`

篩選使用者、 GPU 型號、執行中或排隊中

資料來源

管理員啟用的狀態服務，每 30 秒更新

交自己的程式：把設定寫在腳本裡

國網的 #SBATCH 可以直接用，命令列選項優先

```
#!/bin/bash
#SBATCH -J resnet50
#SBATCH --gres=gpu:v100:2
#SBATCH -c 8 --mem=64G
#SBATCH -t 1-00:00:00
#KJOB --shm 16Gi
set -e
cd /home/jovyan/project
source /home/jovyan/venv/bin/activate
torchrun --standalone \
  --nproc_per_node=2 train.py
```

```
$ ksub train.sh
$ ksub -G 1 train.sh   # 這次只要 1 張
```

✓ 用絕對路徑；程式、資料、輸出放在 /home/jovyan 或群組資料夾

✓ 多張 GPU 要程式支援（torchrun / DDP）

✓ 套件用 pip install --user 或在 /home/jovyan 建虛擬環境

✓ -p 、 -A 、 --mail 等國網專用選項會略過

排隊太久？跑失敗?

先用 kshow NAME 看原因

排隊訊息	怎麼辦
`Insufficient nvidia.com/gpu`	該型號滿了：等，或換有空位的型號
`exceeded quota`	配額用完：關掉閒置的 GPU Notebook
`untolerated taint`	受限節點，沒有權限
`didn't match ... affinity`	-g / -w 名稱錯了，看 ksinfo

失敗訊息	怎麼辦
`exit code 137 OOMKilled`	記憶體不足：加大 --mem
`Bus error / DataLoader`	共享記憶體不足：加大 --shm
`CUDA out of memory`	減小 batch 或換大記憶體 GPU
`DeadlineExceeded`	超過 -t：加大 -t 或續跑

小技巧 -t 時間到或 kcancel 時程式會收到 SIGTERM，有 30 秒存 checkpoint； kacct 會顯示 STOPPED。

讓 AI 助手幫你提交

裝好 kjob 就自動裝好 skill，用一句話描述需求

```
$ kinfo
kjob: installed AI skills in ~/.claude/skills
```

你： 用 kjob 幫我把 train.py 交到有空的 V100， 2 張 GPU、跑 12 小時

AI：
ksinfo 確認 V100 有空 → 檢查路徑與參數 →
`ksub -G 2 -g v100 -t 12h train.sh`
→ 回報工作名稱與日誌位置

skill 規定 AI：

- ✓ 只動你自己的工作，不取消別人的
- ✓ 資源要剛好，不搶整台 GPU
- ✓ 長時間工作一定加 -t 時間上限
- ✓ 刪除多個工作前先問你
- ✓ 提交後回報名稱、型號、日誌路徑

還沒安裝 kjob？ 請 AI 讀 kjob/skills/kjob-install/SKILL.md 照做

指令總整理

kjob 子指令也可以用： kjob submit = ksub 、 kjob sinfo = ksinfo

ksub

背景提交

ksrun

提交並即時看

kqueue

我的工作

kqueue -A

全叢集工作

ksinfo

GPU 空位

kshow NAME

工作詳細

klogs NAME -f

即時輸出

kwait NAME

等待結束

kacct

歷史紀錄

kcancel NAME

取消工作

kexec NAME

進到工作裡

kinfo

我的環境與配額

文件與下一步

`README.md`	簡介、安裝、指令一覽
`TRIAL_GUIDE.md`	10 分鐘新手上手
`docs/USER_GUIDE.md`	所有指令與選項、排錯
`docs/ADMIN.md`	管理員：權限、叢集狀態服務
`kjob/skills/`	AI 助手 skill

github.com/MarkJiang-maji/kjob（branch: cgu-public）· 問題請開 GitHub Issue